

The same information clearly can be obtained from `pg_settings`, with a longer query and with the need to convert the exact value into a human-readable form (note how the memory value is expressed as 8kB chunks):

```
forumbd=> SELECT name, setting, unit FROM pg_settings WHERE name =
'shared_buffers';
      name      | setting | unit
-----+-----+-----
shared_buffers | 16384   | 8kB
(1 row)
```

Therefore, if you need to just glance at a configuration parameter, `SHOW` is the right command to use, while if you need more accurate and complete information about the exact value, where the parameter has been set into the configuration file, and so on, you should query the `pg_settings` catalog.

Finding configuration errors

PostgreSQL provides a very useful catalog called `pg_file_settings` that provides a database administrator with a glance at all the configuration parameters and the file they have been loaded from, thus also providing information about errors. The following query extracts all the information from the catalog, and the trimmed output gives us some important information:

```
postgres=# SELECT name, setting, sourcefile, sourceline, applied, error
FROM pg_file_settings where name = 'log_destination';
-[ RECORD 1 ]-----
name      | log_destination
setting   | stderr
sourcefile | /postgres/16/data/postgresql.conf
sourceline | 444
applied    | f
error      |
-[ RECORD 2 ]-----
name      | log_destination
setting   | csvlog
sourcefile | /postgres/16/data/postgresql.conf
sourceline | 818
applied    | f
error      |
```

```
-[ RECORD 3 ]-----  
name       | log_destination  
setting    | stderr  
sourcefile | /postgres/16/data/postgresql.auto.conf  
sourceline | 4  
applied    | t  
error      |
```

As you can see in the above example installation, the `log_destination` configuration parameter has been loaded multiple times: exactly three, from different source files. The applied one is the configuration settings from the `/postgres/16/data/postgresql.auto.conf` file, on line 4, as reported by the status of the `applied` column. In the case that the parameter contains an error, the `error` column provides a hint about the problem.

Having a parameter defined multiple times within the same file or in different files is not an error, but rather a kind of overwriting of such a parameter. The special view `pg_file_settings` helps in finding out why the parameter defined in a file has not been loaded and which ones have been overwritten by other scattered definitions.

In the case that a parameter cannot be applied because it contains an error, the `error` column will give you a hint about the problem and the `applied` column will result in a false value.

Nesting configuration files

You are not tied to configure every parameter within the `postgresql.conf` file: you can define your own tiny and focused configuration files and instruct PostgreSQL to read and add their content to the configuration. Thanks to this, you can create a set of smaller and cleaner configuration files for specific tasks, so that the maintenance becomes more simple.

PostgreSQL provides three main directives to include other configuration files:

- `include_file`: Includes a single file in the configuration
- `include_dir`: Includes all the files contained in the specified directory
- `include_if_exists`: Includes a file only if it exists

The last directive is very handy because if an included file does not exist, PostgreSQL will throw an error, while with `include_if_exists`, the cluster will not warn you if the file to include has not been created. This is useful for provisioning, for example, where you can set up a main configuration file that includes multiple files and ship those files only to those systems that really require such a configuration.

To give a concrete example, consider a file named `memory.conf` that is placed into the `PGDATA` directory, with the content as follows:

- `shared_buffers = 321 MB`
- `work_mem = 16 MB`

Then add the following directive at the end of the `postgresql.conf` file:

```
include_if_exists = 'memory.conf'
```

In this way, once the cluster is restarted, the `postgresql.conf` file will instrument the cluster to load another configuration file, named `memory.conf`, which in turn will overwrite a couple of memory settings (remember that the last definition of a configuration parameter is the one selected to be applied). The end result can be viewed by querying the `pg_file_settings` catalog:

```
postgres=# SELECT * FROM pg_file_settings WHERE name IN ('shared_buffers',
'work_mem' );
```

setting	sourcefile	applied	error	sourceline	seqno	name
128MB	/postgres/16/data/postgresql.conf	f		127	3	shared_buffers
321MB	/postgres/16/data/memory.conf	t		1	15	shared_buffers
16MB	/postgres/16/data/memory.conf	t		2	16	work_mem

(3 rows)

As you can see, the `shared_buffers` parameter has been overwritten and applied from the freshly created memory file, while the `work_mem`, which is commented out in the default installation, has been applied (without overwriting any other definition) from the included file too.

Configuration contexts

Each configuration parameter belongs to a so-called **context**, a group that defines *when* a change to the parameter can be applied. Several parameters can be changed and take effect during the cluster's life cycle. However, others cannot and require the cluster to be restarted; the context of a configuration parameter helps the system administrator understand when changes will take effect.

Configuration contexts can be extracted from the `pg_settings` catalog, as shown in the following example:

```
forumbd=> SELECT distinct context FROM pg_settings ORDER BY context;
           context
-----
backend
internal
postmaster
sighup
superuser
superuser-backend
user
(7 rows)
```

As you can see, the allowed configuration contexts are as follows:

- **internal:** This configuration value depends on the PostgreSQL source code and is established at compile time, so it cannot be changed unless you decide to compile it from scratch. For example, the size of every memory page is defined in the source code.
- **postmaster:** This process is responsible for getting changes. In other words, the whole cluster (and its main process, `postmaster`) must be restarted to change this.
- **sighup:** With this, the cluster will become aware of changes when given a hang-up signal, typically a reload of the operating system service.
- **superuser-backend** and **backend:** Changes will be applied to both the client and administrator connections. Such changes will be perceivable from the very next connection of either type.
- **user** and **superuser:** These changes will be applied immediately to the current connection, regardless of whether it is an unprivileged connection or a connection from a superuser.

Main configuration settings

PostgreSQL includes a lot of configuration options, and describing all of them here would require an entire book. Moreover, configuration depends on many different factors, including the cluster workload and the connection concurrency. Many parameters can imply different behaviors for other parameters. Therefore, it is not possible to provide a simple and effective step-by-step guide to configuration, but it is possible to provide some suggestions to help you start tuning your cluster.

In the following subsections, you will learn about the main configuration parameters, depending on the main category they belong to. Take your time to clearly understand what every setting does before applying a change, and keep in mind that the configuration contexts could prevent you from seeing immediate results.

WAL settings

WALs are fundamental for the cluster to work properly and to be able to recover from crashes. Therefore, settings related to WALs are vital for the cluster's life cycle.

The main settings are as follows:

- `fsync` tells the cluster to issue an operating system call of `fsync(2)` every time a `COMMIT` is performed; that is, every time something must be stored in the WAL segments.
- `wal_level` indicates the amount of information that the cluster has to keep in the WAL segments.
- `wal_sync_method` tells PostgreSQL which effective `fsync(2)` system call to use.
- `synchronous_commit` tells PostgreSQL whether every `COMMIT` must be followed by an immediate and synchronous `fsync(2)` or whether the flush can be delayed by the time defined in `wal_writer_delay`.
- `wal_writer_delay` and `wal_writer_flush_after` determine how often a process, called the WAL writer, must flush data to disk when operating in asynchronous `COMMIT` mode. After every `wal_writer_delay` worth of milliseconds or after having accumulated `wal_writer_flush_after` megabytes of data, the asynchronous commits are flushed to disk.
- `checkpoint_timeout`, `checkpoint_completion_target`, and `max_wal_size` control checkpointing, as discussed in *Chapter 11, Transactions, MVCC, WALs, and Checkpoints*, when we explained transactions and WALs.

The `fsync` settings must be kept set to on because disabling this will make the cluster subject to data loss: the filesystem will not flush data to disk at the `COMMIT` time, so PostgreSQL has no guarantee that the data has effectively been stored on disk and, in the case of a crash (e.g., power failure), data could be lost. There are very few scenarios when setting this option to `off` makes sense, but keep in mind that (if you do find such a reason) disabling this option will still make your cluster unable to survive a crash.

The `wal_level` setting indicates how much information PostgreSQL must accumulate in the WAL segments. The primary usage of WALs is to make the cluster able to survive a crash, but WALs are also used to propagate changes to other clusters in a replication scenario.

The `wal_level` setting can be set to `minimal`, which is a single cluster with all the information to survive a crash, or `replica` (the default), which makes the WAL also useful for physical replication scenarios, and finally, it can be turned to `logical`, which makes the WALs contain information also for logical replication scenarios.

`wal_sync_method` allows the administrator to configure a specific operating system call to sync dirty buffers. All the POSIX operating systems implement `fsync(2)`, but some of them provide special *flavors* that behave faster or better depending on the filesystem. It is possible to specify the exact name of the system call to use via `wal_sync_method`. Usually, PostgreSQL is shipped with an appropriate configuration for the operating system.

But how can you discover the best (or just the available) `fsync(2)` implementation that fits your operating system? You can launch the `pg_test_fsync` program on your machine to get a good guess about the possible methods you can use, as well as the best one. As an example, on a FreeBSD machine, the program provides the following output:

```
$ pg_test_fsync
5 seconds per test
O_DIRECT supported on this platform for open_datasync and open_sync.

Compare file sync methods using one 8kB write:
(in wal_sync_method preference order, except fdatasync is Linux's default)
      open_datasync                                n/a
      fdatasync                                     6845.727 ops/sec      146 usecs/
op
      fsync                                          3685.769 ops/sec      271 usecs/
op
      fsync_writethrough                            n/a
      open_sync                                     2521.228 ops/sec      397 usecs/
op
...
```

You should compare the available options and choose the fastest one. So, in the preceding example, `wal_sync_method = open_datasync` is the best choice.

`synchronous_commit` is a multiple-choice option that indicates how many WALs have to be written to the disk before returning a “success” state to the transaction issuing the `COMMIT`. By default, this setting has the value `on`, indicating that every single bit must hit the disk before the transaction is allowed to succeed.

Setting the parameter to `off` means that the transaction will succeed even if the WALs have not been flushed to the disk. Unlike the `fsync` setting, turning `synchronous_commit` to `off` is safe and does not provide any data loss, since the asynchronous committing is governed by `wal_writer_delay`, which means the transaction will be consolidated but at a later time. There are also other values for this parameter that all imply an `on` behavior: `local`, `remote_apply`, and `remote_write`. These settings make sense only in a replication scenario when this cluster acts as a primary and is followed (replicated) by one (or more) secondary clusters. In this scenario, `local` means that the transaction will succeed as soon as the primary has flushed every bit to the disk (that is, the default `on` behavior); on the other hand, `remote_write` will wait for the standbys to confirm they have received the same WAL information and are going to replicate it, so it means that the primary has consolidated the transaction and the standbys will soon do the same. Last, `remote_write` will wait for the transaction to be successful until both the primary and the standbys have flushed all the data to disk.

Memory-related settings

PostgreSQL exploits the volatile RAM of the system to cache the data coming from the permanent storage and to manage data that is going to be stored later on.

The main settings related to memory management are as follows:

- `shared_buffers` is the amount of memory PostgreSQL will use to cache data in memory.
- `work_mem` is the amount of memory PostgreSQL will provide, on-demand, to perform particular activities on data.
- `hash_mem_multiplier` is used to determine a threshold about how much memory a connection can use for hash-based operations.
- `maintenance_work_mem` is the amount of memory PostgreSQL reserves for its internal operations.
- `wal_buffers` is the cache used for WAL segments.

`shared_buffers` is probably the most important setting here since it determines the total amount of memory PostgreSQL will use. This memory will be made exclusively available to PostgreSQL and its spawn processes; the memory will not be available to other services running on the same machine. Usually, you should start with a value that is between 25% and 45% of the total RAM your system has. Values that are too low will make PostgreSQL load and flush data from and to the permanent storage, while values that are too high will make PostgreSQL compete with the operating system's filesystem cache, resulting in a possible performance problem.

`work_mem` is the amount of memory that every connection can use to perform a particular data rearrangement, such as what's done in a SORT or a hash join. When dealing with hash-based tasks, a connection is allowed to consume `work_mem * hash_mem_multiplier` memory before the process will swap to disk. In any case, when a process has no way to use any more memory, it will start swapping to disk – for example, converting an in-memory SORT to an on-disk merge SORT.

`maintanance_work_mem` establishes the amount of memory, per session, related to particularly intensive commands such as VACUUM and CREATE INDEX. Since only one of those commands can be active at any moment in a connection, you can raise the value depending on how many administrative connections you are supposed to serve.

`wal_buffers` is probably the easiest setting you can tune with regard to memory: it indicates how much memory to use for caching WAL segments. Since WAL segments are usually written in chunks of 16 megabytes, this is exactly the optimal value for such a setting.

Process information settings

PostgreSQL is a multi-process system, and it spawns a process for serving every incoming connection. There are a couple of settings that can help with monitoring, from the operating system's point of view, every PostgreSQL-related process:

- `update_process_title` makes every process report what it is doing; for example, what query it is executing when asked by operating system tools such as `ps(1)` and `top(1)`.
- `cluster_name` is a mnemonic name used to recognize the cluster that every process belongs to in the case that multiple clusters are running on the same machine.

It is worth noting that these settings could make the system work slower on certain operating systems, such as FreeBSD.

Log-related settings were explained in detail in *Chapter 14, Logging and Auditing*, so they will not be discussed again here.

Networking-related settings

Usually, PostgreSQL listens on a TCP/IP address for incoming connections, which is specified by a bunch of network-related settings. The main settings for this are as follows:

- `listen_addresses` specifies the TCP/IP addresses to listen on.
- `port` specifies the TCP/IP port the postmaster will wait for incoming connections on.

- `max_connections`, `reserved_connections`, and `superuser_reserved_connections` specify the allowed incoming connections.
- `authentication_timeout` and `ssl` indicate the authentication timeout and encrypted mode.

`listen_addresses` can include multiple addresses, separated by a comma, in the case that the server is multi-homed. It can even be specified by the special value `*` to indicate the server should listen on every available address. `port` specifies the TCP/IP port number, which is 5432 by default.

`max_connections` is the max allowance for incoming connections: no more connections will be allowed on the cluster if this threshold is reached. A superuser connection is counted as `superuser_reserved_connection`, and this is due to the fact that, in an emergency, a superuser must still have a way to connect to the cluster. Last, `reserved_connections` counts how many connections are established by users with the special role of `pg_use_reserved_connection`, and indicates a special set of users (not superusers) that have reserved connection slots. Therefore, the number of free connections a normal user can have is `max_connections - superuser_reserved_connection - reserved_connections`.

`authentication_timeout` is the time before an authentication trial will expire, while `ssl` enables the server to handle SSL handshakes on connections (SSL will not be explained here).



There are a lot of parameters to fine-tune SSL and the authentication phase that are not covered in this book since they also require a deep background in those topics.

Archive and replication settings

There are different archiving and replication settings that deal with how the cluster archives its WALs and communicates with other clusters as either a master or a slave. All the settings will be detailed in *Chapter 17, Physical Replication*, and *Chapter 18, Logical Replication*, and they are listed here at a glance:

- `wal_level` (already discussed in a previous section) indicates how the information in the WALs will be used. This can be `minimal` (for a standalone system), `replica` (for a replicated system), or `logical` (for a logical replication).
- `archive_mode`, `archive_command`, `archive_library`, and `archive_timeout` manage the archiving mode – that is, storing WALs to other locations for point-in-time recovery or replication.

- `primary_conninfo` and `primary_slot_name` are used to determine the connection from a standby node to a primary one.
- `hot_standby`, when used on a replicating system, allows for read-only queries.
- `max_standby_archive_delay` and `max_standby_streaming_delay` define the amount of time before a conflicting query on the standby has to be canceled due to some other action that happened on the primary.
- `recovery_min_apply_delay` introduces a delay on the standby node so that it can follow the primary with a timeshift.
- `max_replication_slots` and `max_wal_senders` are used to define how many replication slots will be accepted and in use and how many processes will manage replication.
- `synchronous_standby_names` defines which nodes are to be considered synchronous in replication, and therefore the primary has to receive constant feedback before applying changes.

These settings and other replication-related settings will be discussed in *Chapter 17, Configuration and Monitoring*, and *Chapter 18, Replication*.

Vacuum and autovacuum-related settings

There are different settings that can be used to define and tune the vacuum and autovacuum settings. These were discussed in *Chapter 11, Transactions, MVCC, WALs, and Checkpoints*.

Optimizer settings

The PostgreSQL optimizer is driven by a cost-based approach. It is possible to tune these costs, as discussed in *Chapter 13, Query Tuning, Indexes, and Performance Optimization*.

Statistics collector

PostgreSQL exploits the **statistics collector** to gather facts about what happened in the cluster, as you will learn later in this chapter in the *Monitoring the cluster* section.

Since collecting those numbers has little runtime impact, it is possible to exclude the collection entirely or filter the statistics collector to gather only the facts you are truly interested in. The main settings for this are as follows:

- `track_activities` enables other processes to monitor the current command or query currently being executed.
- `track_counts` gathers counting information about tables and index usage.